
CineBot

Build & Deploy Guide

From a clean machine to a working chatbot in 30 minutes — copy, paste, ship.

Version 1.1 — all known first-run bugs pre-fixed

Target: local Docker Compose deployment

Tested on: Windows 11, macOS 14, Ubuntu 22.04

What's new in v1.1. This guide ships with five bug fixes that the original v1.0 hit on first install: pydantic-settings JSON-decode of CSV env vars, structlog/PrintLogger incompatibility, async SQLAlchemy cross-session ORM detachment in catalogue ingest, missing email-validator dependency for EmailStr, and Celery beat schedule file permissions. All are pre-applied in the source archive.

Contents

0	Prerequisites checklist	3
1	Get the code	5
2	Configure environment	6
3	Build & start the stack	8
4	Initialize data	10
5	Verify & first chat	12
6	Optional enhancements	14
7	Day-to-day operations	16
8	Total reset	17
A	Pre-applied fixes (changelog)	18
B	Troubleshooting decision tree	20

0 Prerequisites checklist

Tick all four before starting. Missing any of them produces obscure errors during build that look unrelated to the missing dependency.

0.1 Docker Desktop — installed and running

- Download from docker.com/products/docker-desktop/ for your OS.
- Launch it. Wait until the whale icon stops animating.
- Verify in a terminal:

```
docker --version
docker compose version
```

Docker version 27.x and Docker Compose version v2.x. Anything else, the install isn't complete.

0.2 Docker Desktop — at least 6 GB of RAM allocated

The stack runs nine containers including ML models. With less than 6 GB Docker silently OOM-kills the API container during startup, which presents as "Restarting (137)" in `docker compose ps`.

- Docker Desktop → gear icon (top right) → **Resources**
- Slide **Memory** to 6 GB or more. 8 GB recommended if you also want Ollama.
- Click **Apply & restart**

0.3 TMDB API key

Free, takes five minutes. Without it, you get a working app with zero movies in it — which works for testing the chat plumbing but isn't useful for actually exploring.

- Sign up at themoviedb.org/signup and verify your email.
- Go to themoviedb.org/settings/api → **Create** → **Developer**.
- Application URL: `http://localhost`. Summary: "personal recommendation project."
- Copy the **API Key (v3 auth)** string — you'll paste it into `.env` in section 2.

0.4 Free local ports

Three ports must be free: 5432 (Postgres), 6379 (Redis), 8000 (API). The most common conflict is a locally-installed Redis or Postgres from a previous project. Quick check on each:

Port	Used by	Test command (Mac/Linux)	Test command (Windows PS)
5432	Postgres	<code>lsof -i :5432</code>	<code>Get-NetTCPConnection -LocalPort 5432</code>
6379	Redis	<code>lsof -i :6379</code>	<code>Get-NetTCPConnection -LocalPort 6379</code>
8000	API	<code>lsof -i :8000</code>	<code>Get-NetTCPConnection -LocalPort 8000</code>
5173	Frontend dev	<code>lsof -i :5173</code>	<code>Get-NetTCPConnection -LocalPort 5173</code>
3001	Grafana	<code>lsof -i :3001</code>	<code>Get-NetTCPConnection -LocalPort 3001</code>

If any port is in use, stop the conflicting service before continuing — most likely culprits are Homebrew Redis, system Postgres, or another Docker project still running.

0.5 Disk space

The build downloads sizeable container images and Python packages.

Item	Size
Docker images (Postgres, Redis, etc.)	~1.5 GB
Backend image (Python + ML deps)	~3.5 GB
Frontend image (Node + Vite)	~600 MB
TMDB catalogue + embeddings	~80 MB in Postgres
Optional: Ollama Llama 3.1 8B model	4.7 GB
Total without Ollama	~6 GB
Total with Ollama	~11 GB

1 Get the code

1.1 Download the v1.1 archive

Use the `cinebot.zip` attached to this guide. It's 110KB and contains the full source tree with the v1.1 fixes already applied.

1.2 Extract and choose a working directory

Pick somewhere stable — you'll be running commands from this directory repeatedly.

Mac / Linux

```
unzip cinebot.zip
cd cinebot
```

Windows (PowerShell)

```
Expand-Archive cinebot.zip -DestinationPath .
cd .\cinebot
```

1.3 Verify the structure

Mac/Linux

```
ls
```

Windows

```
dir
```

You should see: `README.md`, `Makefile`, `docker-compose.yml`, `.env.example`, plus the `backend/`, `frontend/`, `ml/`, `infra/`, `docs/`, `scripts/` folders. If you only see flat files, the archive didn't unzip with its directory structure — re-extract.

Important. All commands in this guide run from the `cinebot/` directory. Stay there. Open a second terminal in the same directory if you want to tail logs while running other commands.

2 Configure environment

Two configuration steps — copy the template, then fill in two values. Total time: ninety seconds.

2.1 Create the .env file

Mac/Linux

```
cp .env.example .env
```

Windows

```
Copy-Item .env.example .env
```

2.2 Edit two values in .env

Open .env in any text editor. Find these two lines and replace the right-hand side:

```
# Replace with the API key from themoviedb.org/settings/api
TMDB_API_KEY=your_tmdb_v3_api_key_here
```

```
# Replace with any random string of 16+ characters
JWT_SECRET_KEY=change-me-to-a-long-random-string-in-prod
```

After editing they should look like:

```
TMDB_API_KEY=abc123def456ghi789jkl012mno345pq
JWT_SECRET_KEY=local-dev-secret-cinebot-1234567890ab
```

Save and close the file.

Heads up. Don't add quotes around the values, don't add spaces around the =, don't leave the line commented (no leading #). All three are common reasons the env var fails to load.

2.3 Choose your LLM strategy

CineBot uses an LLM to polish the wording of recommendations. The core experience works without one. You have three options — pick Option A for first run.

Option	Setup	Cost	Speed
A — None	Already configured (default)	Free	Fast — template responses
B — OpenAI	Set OPENAI_API_KEY in .env	~\$1/wk	Fast — cloud inference
C — Ollama	4.7 GB local model download	Free	Slow on CPU

If you choose Option B (OpenAI) — find these two lines in .env and edit them:

```
LLM_PROVIDER=openai
OPENAI_API_KEY=sk-your-openai-key-here
```

Get a key from platform.openai.com/api-keys. Requires a billing method on the OpenAI account; gpt-4o-mini costs about \$0.15 per million input tokens — heavy use is well under \$1/week.

If you choose Option C (Ollama) — leave the LLM_PROVIDER as ollama in .env. The Ollama service stays profile-gated by default; section 6 covers enabling it.

3 Build & start the stack

Two commands. The first one takes the longest because Docker has to download images, install Python ML libraries, and set up everything from scratch.

3.1 Build & start everything

```
docker compose up --build -d
```

First run takes 5–15 minutes — most of that is pip-installing PyTorch, Transformers, sentence-transformers, and spaCy. Subsequent runs of the same command finish in under 30 seconds because Docker caches the layers.

Output you'll see (final part):

```
[+] Building 287.4s (23/23) FINISHED
=> [api builder 4/5] RUN pip install ... ✓ DONE
=> [api dev 3/3] COPY ... ✓ DONE
[+] Running 9/9
✓ Container cinebot-postgres      Healthy
✓ Container cinebot-redis         Healthy
✓ Container cinebot-mlflow        Started
✓ Container cinebot-prometheus    Started
✓ Container cinebot-grafana       Started
✓ Container cinebot-api           Started
✓ Container cinebot-worker        Started
✓ Container cinebot-beat          Started
✓ Container cinebot-frontend      Started
```

3.2 Confirm everything is running

```
docker compose ps
```

Nine rows. The stateful services (postgres, redis) should show (healthy). The API will first show (health: starting) for ~30 seconds while it loads spaCy and the embedder, then flip to (healthy).

3.3 Watch logs while you wait

Open a second terminal in the same cinebot/ directory and run:

```
docker compose logs -f api
```

You'll see startup events: app_starting, embedder_loading, embedder_ready, app_ready. Press **Ctrl+C** to stop tailing — this doesn't stop the service, just stops watching logs.

3.4 Verify the API is reachable

```
curl http://localhost:8000/health
```

`{"status": "ok"}` — the API is up.

If the API container is restart-looping (status shows Restarting (1) repeatedly): run `docker compose logs api --tail 80` and look at the bottom of the output. The most common cause is running an older v1.0 archive instead of v1.1. Re-download the v1.1 zip and start over.

4 Initialize data

The API is running but Postgres is empty — no schema, no movies, no embeddings. These three commands populate it. Run them in order.

4.1 Apply the database schema

```
docker compose exec api alembic upgrade head
```

Three log lines confirming the migration ran. Takes 2 seconds.

Output:

```
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 20260423_0001, initial schema with pgvector
```

4.2 Ingest the TMDb movie catalogue

```
docker compose exec api python -m app.scripts.ingest_tmdb
```

Takes 3–5 minutes. Pulls 50 pages of popular movies (~1000 films) from TMDb, plus the genre list. Idempotent — safe to re-run if interrupted.

Output streams as it goes:

```
[info] genres_ingested count=19
[info] page_ingested page=1 count=20 total=20
[info] page_ingested page=2 count=20 total=40
...
[info] page_ingested page=50 count=20 total=1000
[info] ingest_complete total=1000 pending_embeddings=1000
```

If you see `tmdb_api_key_missing` — your `.env` file doesn't have the key set. Edit `.env`, add the key, then `docker compose restart api celery-worker celery-beat` and re-run the `ingest` command.

4.3 Generate semantic embeddings

```
docker compose exec api python -m app.scripts.build_embeddings
```

Takes 2–4 minutes. Loads the sentence-transformer model (~90 MB download on first use), then encodes each movie's title + tagline + overview + genres into a 384-dimensional vector. Stores them in the pgvector HNSW index for fast semantic search.

Output:

```
[info] embedder_loading model=sentence-transformers/all-MiniLM-L6-v2
[info] embedder_ready dim=384
[info] batch_embedded count=64 total=64
[info] batch_embedded count=64 total=128
...
[info] embed_complete total=1000
```

4.4 Sanity-check the data

Open a Postgres shell inside the container and verify:

```
docker compose exec postgres psql -U cinebot -d cinebot
```

In the psql prompt, run:

```
-- count movies (should be ~1000)
SELECT COUNT(*) FROM movies;

-- five most popular
SELECT title, vote_average FROM movies
  ORDER BY popularity DESC LIMIT 5;

-- count of embedded movies (should equal total)
SELECT COUNT(*) FROM movies WHERE embedding IS NOT NULL;

-- exit
\q
```

5 Verify & first chat

5.1 Open the chat UI

In your browser:

`http://localhost:5173`

CineBot's auth page — charcoal background, amber accent, a Sign-In / Create toggle. If it doesn't load, give the frontend container 30 more seconds (it's the last to start).

5.2 Create an account

- Click **Create** (the right half of the toggle).
- Email: anything that looks like an email. `test@example.com` works — it's local only.
- Password: at least 8 characters.
- Display name: optional.
- Click **Create account**.

You'll be dropped onto the chat page. The sidebar shows your email and a green "LIVE" status — that's the WebSocket connection.

5.3 Have a real conversation

Try this five-message script that exercises every feature:

MESSAGE 1 — RECOMMEND

recommend a feel-good comedy from the 90s

Five movie cards stream in within 1–3 seconds. Each card shows poster, title, year, score chip in amber, and 2–3 reasons. Click  on one of them — the button turns amber with a check.

MESSAGE 2 — REFINE

more like the second one

New slate of five — the recommender averages the embeddings of the previous slate's top 3 films and re-runs retrieval with those films excluded.

MESSAGE 3 — GREETING

hey

Short greeting back, no recommendations.

MESSAGE 4 — OUT-OF-SCOPE

what's the weather tomorrow?

"I only do movie recs, but I do them well. Give me a mood, a genre, or a vibe."

MESSAGE 5 — GOODBYE

thanks, bye

"Enjoy the movie!" or similar — the system gracefully closes the conversation.

Why these exact phrasings? Without training the intent classifier (section 6.1), CineBot uses a rule-based fallback that looks for keywords like *recommend*, *suggest*, *watch*, *in the mood*. Phrasings like *"feel-good comedy from the 90s"* without those keywords get classified as out-of-scope. Train the model (section 6.1) to handle natural phrasing; otherwise stick to the verb forms above.

6 Optional enhancements

Everything in section 5 works on the rule-based defaults. The four enhancements below take the system from "working" to "production-quality." All optional. Skip any.

6.1 Train the DistilBERT intent classifier

With the trained model, CineBot understands natural phrasing ("feel-good comedy from the 90s" alone, no "recommend" keyword required). Training uses the seed dataset of ~210 labelled examples shipped with the repo.

```
docker compose exec api python -m app.scripts.train_intent
```

Runs for ~5 minutes on CPU. Logs go to MLflow at <http://localhost:5000>. After it finishes, the model is saved to /app/ml/models/intent_classifier/. On the next API restart, it auto-detects the model and uses it.

```
docker compose restart api
```

In the API logs you'll now see `intent_using kind=transformer` instead of `kind=rule_based`.

6.2 Enable Ollama for local LLM polish

The Ollama service is profile-gated — it doesn't start by default, to save you a 4.7 GB model download. Two extra commands enable it:

```
docker compose --profile ollama up -d ollama
docker compose exec ollama ollama pull llama3.1:8b
```

First command starts the Ollama container (a few seconds). Second command downloads ~4.7 GB. Total elapsed: depends on your bandwidth.

Then verify `.env` has:

```
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://ollama:11434
OLLAMA_MODEL=llama3.1:8b
docker compose restart api celery-worker
```

6.3 Switch to OpenAI for LLM polish

Faster than Ollama, costs pennies. Edit `.env`:

```
LLM_PROVIDER=openai
OPENAI_API_KEY=sk-your-key-here
docker compose restart api celery-worker
```

6.4 Browse Grafana dashboards

The pre-provisioned dashboard shows real-time metrics — request rate, p95 latency, recommendation count, feedback like-ratio. Visible only after you've made some requests.

- Open <http://localhost:3001>
- Username `admin` / password `admin`
- Skip the password change prompt (or change if you want).
- Sidebar → **Dashboards** → **CineBot** folder → **CineBot — Service Overview**

6.5 Explore the API directly

Interactive Swagger UI at:

`http://localhost:8000/docs`

To authenticate: call POST `/api/v1/auth/login` with your credentials, copy the `access_token` from the response, click the **Authorize** button at the top, paste.

7 Day-to-day operations

Once everything is built and seeded, day-to-day commands are short. Reference table:

Action	Command
Start the stack	<code>docker compose up -d</code>
Stop (keep data)	<code>docker compose down</code>
Tail API logs	<code>docker compose logs -f api</code>
Tail all logs	<code>docker compose logs -f</code>
List services	<code>docker compose ps</code>
Restart only the API	<code>docker compose restart api</code>
Open a Postgres shell	<code>docker compose exec postgres psql -U cinebot -d cinebot</code>
Open a bash shell in API container	<code>docker compose exec api bash</code>
Re-apply migrations	<code>docker compose exec api alembic upgrade head</code>
Re-ingest TMDb	<code>docker compose exec api python -m app.scripts.ingest_tmdb</code>
Re-build embeddings	<code>docker compose exec api python -m app.scripts.build_embeddings</code>
Train the intent classifier	<code>docker compose exec api python -m app.scripts.train_intent</code>
Run unit tests	<code>docker compose exec api pytest -v</code>

Useful URLs

URL	Purpose
<code>http://localhost:5173</code>	Chat UI
<code>http://localhost:8000/docs</code>	Interactive API documentation (Swagger)
<code>http://localhost:8000/health</code>	Liveness probe
<code>http://localhost:8000/ready</code>	Readiness probe (checks DB + Redis)
<code>http://localhost:8000/metrics</code>	Prometheus metrics scrape endpoint
<code>http://localhost:3001</code>	Grafana (admin/admin)
<code>http://localhost:9090</code>	Prometheus
<code>http://localhost:5000</code>	MLflow

8 Total reset

If anything gets into a bad state — schema drift, corrupted embeddings, ports stuck, mystery errors — there's no shame in wiping and starting over. The data is recreatable in 10 minutes and a fresh build is guaranteed-clean.

8.1 Stop and wipe everything

```
docker compose down -v
```

Stops all containers AND deletes the named volumes (postgres-data, redis-data, mlflow-data, etc.). The -v flag is the destructive part.

8.2 Force a clean image rebuild

Optional, but useful if you suspect Docker layer cache is masking a fix you applied to `pyproject.toml` or a Dockerfile.

```
docker compose build --no-cache api
```

Takes 5–8 minutes — pip-installs every Python dependency from scratch, no cache. Only do this if you've changed pyproject and a normal rebuild didn't pick it up.

8.3 Re-run the full setup

From the `cinebot/` directory:

```
docker compose up --build -d                # build & start
docker compose exec api alembic upgrade head # schema
docker compose exec api python -m app.scripts.ingest_tmdb # ~3-5 min
docker compose exec api python -m app.scripts.build_embeddings # ~2-4 min
```

Total time: about 10 minutes (most of which is unattended ingestion). User accounts and chat history are gone — you'll register again at `localhost:5173`.

8.4 Free up disk space (after wipe)

To reclaim disk space from old images and dangling volumes:

```
docker system prune -a
```

Type y to confirm. Removes all stopped containers, unused networks, and untagged images. Does NOT remove volumes still in use.

A Pre-applied fixes (changelog)

Five bugs surfaced during the first end-to-end run of v1.0. All are fixed in v1.1. This appendix exists for two reasons: transparency about what changed, and to help you understand the shape of issues to look for if you modify the code.

A.1 pydantic-settings JSON-decode of CSV env vars

Symptom: `json.decoder.JSONDecodeError: Expecting value: line 1 column 1` on every API call. **Cause:** `cors_origins: list[str]` in `app/core/config.py` made pydantic-settings v2 try to JSON-decode the comma-separated env value before any custom validator ran. **Fix:** changed type to `str | list[str]` — pydantic-settings treats the union as "simple" and the comma-split validator now runs. **File:** `backend/app/core/config.py`.

A.2 structlog add_logger_name + PrintLogger incompatibility

Symptom: `AttributeError: 'PrintLogger' object has no attribute 'name'` on the first log line emitted. **Cause:** `add_logger_name` reads `.name` on the logger, which `PrintLogger` doesn't expose. **Fix:** removed `add_logger_name` from the processor chain — event field already carries the context. **File:** `backend/app/core/logging.py`.

A.3 Cross-session ORM detachment in TMDb ingest

Symptom: `sqlalchemy.exc.InvalidRequestError: Object <Genre> is already attached to session '1' (this is '2')` on the first page of movie ingestion. **Cause:** Genre ORM objects loaded in session #1 were referenced from session #2 (one session per page). Async SQLAlchemy is strict about cross-session object reuse. **Fix:** passed only the genre IDs (plain ints) between sessions; re-fetched the Genre rows in each per-page session before linking them to movies. **File:** `backend/app/scripts/ingest_tmdb.py`.

A.4 Missing email-validator dependency

Symptom: `ImportError: email-validator is not installed` — API container restart-loops at startup. **Cause:** `EmailStr` in Pydantic v2 requires the optional `email-validator` package, which isn't a default dependency. **Fix:** declared `pydantic[email]>=2.9.0` instead of plain `pydantic>=2.9.0`. Also pinned `bcrypt<5.0.0` defensively against a known passlib incompatibility. **File:** `backend/pyproject.toml`.

A.5 Celery beat permission denied on schedule file

Symptom: `[Errno 13] Permission denied: 'celerybeat-schedule'` — beat container restart-loops, but worker stays healthy. **Cause:** beat persists its schedule to a file in the working directory, where the non-root container user can't write. **Fix:** added `--schedule=/tmp/celerybeat-schedule` flag to the beat command. **Impact when present:** beat crash-looped, but the only periodic task it runs is hourly embedding backfill — no user-visible impact. **File:** `docker-compose.yml`.

B Troubleshooting decision tree

Decision tree for the most common issues. Find the symptom that matches yours and follow the chain.

Symptom: docker compose up fails with port conflict

"Bind for 0.0.0.0:6379 failed: port is already allocated" (or 5432, or 8000). Something else on your machine is using the port. Either stop that thing, or change the host-side port in `docker-compose.yml`. On Mac, `brew services list` shows what's running. On Linux, `sudo systemctl list-units --type=service --state=running`. On Windows, `Get-Service | Where {$_.Status -eq 'Running'}`.

Symptom: API container keeps restarting

Run `docker compose logs api --tail 80` and read the *bottom* of the output (the most recent crash). Common reasons: (1) wrong `.env` file — `JWT_SECRET_KEY` too short or missing; (2) running v1.0 instead of v1.1 — re-download; (3) `ImportError` from a missing package — most are fixed in v1.1.

Symptom: registration says "Failed to fetch"

This is a browser-level error meaning the request never reached the API. Check `docker compose ps` — is `cinebot-api` healthy? If yes, open browser DevTools (F12) → Network tab → try registering → click the failed request → check Status. If it's `net::ERR_EMPTY_RESPONSE`, the API is restart-looping. If it's a CORS error, see B.4 below.

Symptom: CORS error in browser console

"No 'Access-Control-Allow-Origin' header is present on the requested resource". Confirm your `.env` contains: `CORS_ORIGINS=http://localhost:5173,http://localhost:3000`. Then `docker compose restart api`.

Symptom: registration succeeds but recommendations are empty

You skipped section 4.3 (build embeddings). Run `docker compose exec api python -m app.scripts.build_embeddings`.

Symptom: every message gets classified as out-of-scope

You're on the rule-based intent fallback. Either train the model (section 6.1) or rephrase using verbs like *recommend*, *suggest*, *watch*, or *in the mood*.

Symptom: rate limit exceeded on every request

Redis lost state, or you hammered the API in development. Run `docker compose exec redis redis-cli FLUSHDB` to clear the rate-limit buckets. This does not affect sessions or movie data — those live in Postgres.

Symptom: tests fail with database connection refused

If you ran `docker compose exec api pytest` and the integration tests fail, that's expected — the integration suite is scaffolded but not populated in v1.1. Run only unit tests: `docker compose exec api pytest -m unit -v`.

Still stuck? Three things to gather and ask for help with:

- Output of `docker compose ps`
- Last 80 lines of `docker compose logs api --tail 80`
- The exact command you ran and the error message you got

Now go watch something good.